

Pruebas y evaluación

Contenidos

1. Introducción
2. Tipos de pruebas
 - 2.1 – Pruebas unitarias
 - 2.2 – Pruebas de integración
 - 2.3 – Pruebas de regresión
 - 2.4 – Pruebas de estrés
 - 2.5 – Pruebas de seguridad
3. Documentación y plan de pruebas

1. Introducción

Las pruebas durante el desarrollo de un producto software son un paso fundamental para validar su construcción. Estas pruebas deben ir mucho más allá que los simples casos de prueba que puede realizar un programador durante la fase de construcción. La gestión y la automatización de estos casos de prueba son algo fundamental para poder asegurar que el software cumple con los requisitos que se marcaron al comienzo de su desarrollo. Una persona puede ejecutar todas las pruebas, pero sería una tarea muy costosa y contraproducente. Como humanos, contamos con una capacidad limitada para realizar una gran cantidad de acciones de forma repetible y fiable. Sin embargo, una máquina puede hacerlo rápidamente y comprobará que la combinación de usuario y contraseña para iniciar sesión funciona hasta 100 veces sin quejarse.

2. Tipos de pruebas

De forma muy general se puede hablar de dos grandes tipos de pruebas. Las **pruebas de caja negra** son las que evalúan la aplicación desde un punto de vista externo sin preocuparnos por el interior de esta. Son las utilizadas para evaluar las interfaces de usuario en términos de usabilidad,

accesibilidad, etc. Las **pruebas de caja blanca** se basan en la evaluación del código interno del software.

En base a su forma de ejecutarse podemos también distinguir dos tipos de pruebas: **manuales** y **automatizadas**. Las pruebas manuales se realizan en persona, haciendo clic a través de la aplicación o interactuando con el software y las API con las herramientas adecuadas. Resultan muy costosas, ya que requieren que alguien configure un entorno y ejecute las pruebas, y pueden ser propensas a errores humanos, ya que el encargado puede cometer erratas u omitir pasos en el script de la prueba. Por otro lado, las pruebas automatizadas se realizan a través de una máquina que ejecuta un script de prueba escrito con antelación. Estas pruebas pueden variar mucho en cuanto a complejidad, desde comprobar un único método en una clase hasta comprobar que al realizar una secuencia de acciones complejas en la IU se generan los mismos resultados. Las pruebas automatizadas son mucho más potentes y fiables que las manuales, pero su calidad depende de lo bien que se hayan escrito los scripts de las pruebas.

De forma mucho más particular, normalmente cuando hablamos de pruebas hemos de concretar el tipo de estas. Vamos a resumir los principales tipos de pruebas que podemos realizar sobre un producto software:

- **Pruebas unitarias:** Las pruebas unitarias son de muy bajo nivel, como la fuente de tu aplicación. Están compuestas de pruebas de métodos y funciones individuales de las clases, componentes o módulos que usa tu software. En general, las pruebas unitarias son bastante baratas de automatizar, y se pueden ejecutar rápidamente mediante un servidor de integración continua.
- **Pruebas de integración:** Las pruebas de integración verifican que los distintos módulos o servicios utilizados por tu aplicación funcionan bien en conjunto. Por ejemplo, se puede probar la interacción con la base de datos o asegurarse de que los microservicios funcionan bien en conjunto y según lo esperado. Estos tipos de pruebas son más costosos de ejecutar, ya que requieren que varias partes de la aplicación estén en marcha.
- **Pruebas funcionales:** Las pruebas funcionales se centran en los requisitos empresariales de una aplicación. Solo verifican el resultado de una acción y no comprueban los estados intermedios del sistema al realizar dicha acción. A veces, se confunden las pruebas de integración con las funcionales, ya que ambas requieren que varios componentes interactúen entre sí. La diferencia es que una prueba de integración puede simplemente verificar que puedes hacer consultas en la base de datos, mientras que una prueba funcional esperaría obtener un valor específico desde la base de datos, según dicten los requisitos del producto.
- **Pruebas integrales:** Las pruebas integrales replican el comportamiento de un usuario con el software en un entorno de aplicación completo. Además, verifican que diversos flujos de usuario funcionen según lo previsto, y pueden ser tan sencillos como cargar una página web o iniciar sesión, o mucho más complejos, como la verificación de notificaciones de correo electrónico, pagos en línea, etc. Las pruebas integrales son muy útiles, pero son costosas de llevar a cabo y pueden resultar difíciles de mantener cuando están automatizadas. Se recomienda tener algunas pruebas integrales clave y depender más de pruebas de menor nivel (unitarias y de integración) para poder detectar rápidamente nuevos cambios.
- **Pruebas de regresión:** Verifican un conjunto de escenarios que funcionaron correctamente en el pasado, para asegurar que continúen así. Una falla en una prueba de regresión significa que una nueva funcionalidad ha afectado otra funcionalidad que era correcta en el pasado, causando una «regresión».
- **Pruebas de rendimiento:** Las pruebas de rendimiento comprueban el comportamiento del sistema cuando está sometido a una carga importante. Estas pruebas no son funcionales y pueden tener la forma variada de entender la fiabilidad, la estabilidad y la disponibilidad de la plataforma; por ejemplo, puede estar observando tiempos de respuesta al ejecutar un gran número de solicitudes, o cómo se comporta el sistema con una cantidad significativa de datos. Por su naturaleza, las pruebas de rendimiento pueden ser bastante costosas de implementar y ejecutar,

pero pueden ayudarte a entender si los nuevos cambios van a degradar tu sistema.

- **Pruebas de seguridad:** Hoy en día son de vital importancia junto con las de rendimiento y permiten evaluar y detectar posibles vulnerabilidades que nuestro sistema puede contener y que podrían ser explotadas en el futuro por un atacante.
- **Pruebas de usuario:** Son las ejecutadas por usuarios reales del contexto en el que funcionará la aplicación. Normalmente se realizan audiadas por expertos que deberán evaluar la «experiencia el usuario» y retroalimentar con las conclusiones para realizar cambios en futuras versiones.

El objetivo final de las pruebas será el de generar retroalimentación hacia el equipo de desarrollo para realizar cambios en futuras versiones. Normalmente, la primera versión a la que se realiza un plan de pruebas completo suele llamarse **versión Alfa** y acaba siendo probada por un número muy reducido de usuarios. De ella, se extrae una versión posterior llamada **versión Beta** que suele ser una versión completa casi al 100 % y cuyas pruebas suelen dejarse a un grupo mucho mayor de usuarios llamados comunmente «beta testers».

Como se ha visto, hay numerosos tipos de pruebas en el proceso completo de desarrollo de software y todas ellas no han de ser ejecutadas ni en mismo número ni con la misma frecuencia. El diagrama de la figura 1 ilustra la distribución de las pruebas relacionadas con la frecuencia de cada una de ellas en el proceso de desarrollo.



Figura 1: Diagrama que representa la frecuencia de cada tipo de prueba

2.1. Pruebas unitarias

El testing o prueba de código hace referencia a los procesos de validación del funcionamiento del software de un determinado programa o aplicación. Este mecanismo se caracteriza por proporcionar una garantía de calidad del sistema, para lo que utiliza recursos como las pruebas unitarias de software, que se encargan de comprobar que un fragmento del código fuente esté funcionando de forma adecuada.

Este tipo de pruebas son de vital importancia para la detección de errores, ya que, sin este testeo, no podrían identificarse hasta fases más avanzadas del desarrollo, como, por ejemplo, la fase de integración. Esto implica que las pruebas unitarias de software **evitan la escalada de errores en el código al identificarlas de manera temprana**. Suelen llevarse a cabo en una fase temprana del desarrollo. Estas pruebas unitarias son muy sencillas de hacer, consumirán poco tiempo de nuestro trabajo y por contra nos permitirán asegurar con bastante margen el buen funcionamiento de las clases que estamos empleando. Idealmente, cuando planeamos y escribimos pruebas unitarias, debemos aislar la funcionalidad hasta un punto en que no se pueda desglosar más, y entonces escribir pruebas a partir de ello. Justamente, el nombre de este tipo de *testing* hace referencia a una «unidad de código», que es

independiente del resto. Estas pruebas verifican que el nombre de la función o método sea adecuado, que los nombres y tipos de los parámetros sean correctos, y así mismo el tipo y valor de lo que se devuelve como resultado.

Para realizar un test automático de prueba, debemos realizar una clase o un trozo de código que se encargue de hacer las instancias de las clases que queremos probar y que se encargue de llamar a los métodos de dichas clases, pasando parámetros concretos y viendo que el resultado es el esperado. Es posible que dicho resultado nos venga por el return del método, o es posible que ese método no devuelva nada, pero haga algo en otro sitio y es ahí donde debemos ir a verificar si se ha hecho. Por ejemplo, un método `suma()` al que se pasan dos parámetros, devuelve un resultado, es fácil de testear. Un método `insertaEnBaseDeDatos()` al que se pasan varios parámetros, no devuelve nada (quizás un `true` o `false` si la operación ha tenido éxito), será más complicado de probar, pues nuestro programa de pruebas tendrá que ir a la base de datos y comprobar que se ha hecho la inserción.

Nuestro programa de prueba puede ser simplemente una clase con un `main()` que haga eso y que de alguna forma devuelva si la clase funciona o no. Un buen programa de prueba automático no debe darnos largos listados en los que nosotros analizamos manualmente si los resultados son los esperados o no. Tampoco debe requerir que pulsemos botones, pulsemos «intro» o hagamos cualquier otra cosa mientras se hace la prueba. Un buen programa automático de prueba debe arrancarse de forma simple, debe pasar él solo todas las pruebas y únicamente nos debe decir si todas están bien o no. En caso de estar mal, debe darnos información de qué prueba es la que ha fallado.

Dado que las pruebas unitarias no deben tener ningún tipo de dependencia, se suele reemplazar los llamados a APIs y servicios externos por funcionalidad que los imite (para que no exista interacción que vaya más allá de la unidad que está siendo probada). En muchos casos inclusive se suele reemplazar las consultas a bases de datos, de modo que la prueba se enfoque en operar a partir de los valores de entrada, sin depender de ninguna fuente externa. Si no es factible aislar el uso de bases de datos de nuestras pruebas unitarias, será importante tener en cuenta el rendimiento y buscar optimizar nuestras consultas. Esto es importante, porque si nuestras pruebas unitarias son de larga duración, resultará incómodo ejecutarlas y ralentizará significativamente los tiempos de desarrollo.

2.1.1. Uso de JUnit

Desde hace años en el ecosistema Java se viene utilizando el famoso framework de pruebas JUnit. Este paquete de clases permite automatizar cómodamente las pruebas unitarias que les queramos pasar a nuestras clases e incluso asociar su arranque con Ant.

El funcionamiento básico de JUnit consiste en crear una clase que suele llamarse igual que la clase que queremos testear pero con el prefijo `Test`. Esta clase tendrá ciertos métodos sobrescritos en los que incluiremos nuestras acciones y que serán ejecutados por JUnit en el momento de lanzar nuestros tests. Estos métodos además incorporan anotaciones Java que le indican a JUnit cual es su función. Existen muchas anotaciones interpretadas por JUnit pero vamos a ver solo algunas de ellas:

- **@Test:** Esta anotación describe que ese método es un test que se ha de lanzar por JUnit
- **@Before:** Este método se invoca siempre antes de la ejecución de cada método de test de la clase.
- **@After:** Este método se invoca siempre después de la ejecución de cada método test.
- **@BeforeClass:** Este método se invoca siempre antes de la ejecución de todos los métodos de test. En este método es útil hacer las operaciones más tediosas o pesadas en tiempo como las conexiones a las bases de datos u otras comprobaciones.

Dentro de los métodos de test realizaremos las invocaciones de los métodos de la clase a testear que deseemos probar. Para comprobar que el método nos ha devuelto algo esperado o no usaremos un

método de tipo **Assert** también llamados comunmente «aserciones». Estos métodos permite que JUnit realice la comprobación comparando el resultado obtenido con el esperado. La lista de las principales aserciones de JUnit es:

- `assertArrayEquals()`
- `assertEquals()`
- `assertTrue()`
- `assertFalse()`
- `assertNull()`
- `assertNotNull()`

Vamos a ver un ejemplo sencillo en el que detallaremos solo las partes de código más importantes.★ Hemos creado una pequeña clase llamada **Calculadora** que permite evaluar expresiones de sumas, restas y multiplicaciones de varios operandos que se operen siempre con un mismo operador, nunca con operadores combinados. Aquí podemos ver un borrador de la interfaz de nuestra clase con posibles ejemplos que debería interpretar:

```
public class Calculadora {  
  
    public int suma(String expression) throws NumberFormatException{  
        ...  
    }  
  
    public int resta(String expression) throws NumberFormatException{  
        ...  
    }  
  
    public int mult(String expression) throws NumberFormatException{  
        ...  
    }  
}  
  
Ejemplos de uso:  
2+3  
4-5-10  
9*2*3  
7+3+1+1+9  
...
```

Evidentemente podemos crear un método **main** donde evaluemos todos estos métodos pero ¿qué ocurre si esta clase forma parte de un proyecto mucho más grande con un método principal ya creado? La forma más útil y limpia de probar esta clase de forma aislada dentro del propio proyecto es crear una prueba unitaria solo para ella usando JUnit. Aquí vemos un resumen de la clase **TestCalculadora** que probará la clase **Calculadora**.

```

public class TestCalculadora {

    @Test
    public void evaluaSumas() throws NumberFormatException{

        Calculadora c = new Calculadora();
        int res = 0;

        res = c.suma("1+2");
        assertEquals(3, res);

        res = c.suma("6+2+8");
        assertEquals(16, res);

        res = c.suma("5+5+4+10");
        assertEquals(24, res);
    }

    @Test
    public void evaluaRestas() throws NumberFormatException{
        ...
    }

    // otros tests
}

```

Podemos evaluar también si un método lanza una excepción y comprobar así que la salida errónea que esperábamos se ha producido. Esto puede hacerse usando un añadido a la anotación `Test` tal y como se ve en el ejemplo siguiente:

```

@Test(expected = NumberFormatException.class)
public void evaluaSumasReales() throws NumberFormatException{

    Calculadora c = new Calculadora();
    int res = 0;

    res = c.suma("1+2.5");
}

```

Para terminar y lanzar nuestros test usaremos lo aprendido de Ant. Añadiremos al fichero `build.xml` un nuevo target con la receta para que Ant sepa lanzar JUnit y arrancar nuestros test. De esta forma, para arrancar las pruebas bastará con que usemos el comando `ant test` en nuestro intérprete de órdenes.

```

<target name="test" depends="compila">
    <junit printsummary="on" haltonfailure="yes" fork="true">
        <classpath>
            <path refid="classpath.app"/>
        </classpath>
        <formatter type="brief" usefile="false" />
        <batchtest>
            <fileset dir="${srcdir}" includes="**/Test*" />
        </batchtest>
    </junit>
</target>

```

2.2. Pruebas de integración

Este tipo de pruebas evalúa la forma en que interactúan y operan varios módulos de aplicaciones de software de forma cohesiva. El sistema se divide en componentes conocidos como módulos o unidades. Cada módulo es responsable de una tarea específica. El verdadero desafío llega cuando combinamos estos componentes para desarrollar todo el sistema de software. En esta etapa, se comienza a examinar cuidadosamente las conexiones entre cada módulo para descubrir cualquier problema potencial que resulte de una sola unidad. Cuando las pruebas han finalizado, se realizan pruebas de punta a punta para evaluar la funcionalidad de la aplicación de principio a fin. Los módulos del software son combinados manualmente y se evalúan sus relaciones en las pruebas manuales de componentes. Las pruebas manuales pueden ser costosas y vulnerables al error humano. Puede resultar desafiante cubrir adecuadamente todas las posibilidades de integración en sistemas complicados, por tanto, para abordar estos problemas se suele emplear la automatización. Existen dos grandes tipologías de pruebas de integración:

- **Pruebas incrementales:** Las pruebas incrementales consisten en probar módulos de software en pequeños incrementos. Las pruebas de software comienzan con partes más pequeñas y avanzan progresivamente a través de todo el sistema. Cada prueba mejora el software al integrar los módulos adicionales. En comparación con probar el sistema completo de forma simultánea, esto ofrece ventajas, incluyendo una retroalimentación temprana, una resolución de problemas más directa y de menor complejidad.
- **Pruebas no incrementales:** Este tipo de pruebas implica compilar todos los módulos de software en una estructura y evaluarla como una unidad. Los módulos individuales no se examinan por separado. Se combinan y prueban en un único proceso. Requiere una comunicación firme entre los equipos de desarrollo y de pruebas para identificar y resolver adecuadamente cualquier problema detectado durante el procedimiento de prueba. La estrategia big-bang puede ser más rápida y menos costosa que otros enfoques alternativos, ya que los desarrolladores no necesitan pruebas incrementales. Esta estrategia podría funcionar para sistemas de software con menos relaciones entre componentes y menos complejas. Sin embargo, puede resultar difícil identificar el módulo preciso cuando se descubren fallos durante las pruebas.



Un problema muy común en las pruebas de integración es que hacer con las dependencias del módulo o módulos que vamos a probar y que no están disponibles: bases de datos, servicios en producción, etc. Debemos crear algún tipo de simulación de estos recursos que «engañe» a los componentes probados a creer que están ahí. Los dos mecanismos más utilizados para ello son los llamados *stubs* y los *mockers*. No son exactamente lo mismo pero tampoco vamos a entrar en detalles para explicar sus diferencias. Veamos un ejemplo sencillo para entender ambos conceptos.

Supongamos que estamos probando una clase llamada **CalculadoraNotas** que calcula la nota media de una clase de estudiantes. En este caso, utilizaremos un *stub* para simular el comportamiento de una dependencia externa que proporciona las notas de los estudiantes. Primero, definimos una interfaz **IServicioNotas** que representa el servicio que obtiene las notas de los estudiantes. Esto puede no ser necesario pero nos aporta limpieza a la hora de conectar el componente simulado y el real. Tras esto implementamos una clase llamada **ServicioNotasStub** que simula ser el servicio de notas real que consulta la base de datos pero realmente no lo es. Simplemente retorna un array predefinido de notas que nos hemos inventado.

```

----- Fichero IServiciosNotas.java -----

public interface IServicioNotas {
    double[] obtenerNotas();
}

----- Fichero ServicioNotasStub.java -----

public class ServicioNotasStub implements IServicioNotas {
    @Override
    public double[] obtenerNotas() {
        // Devolvemos un conjunto de notas simuladas
        // en lugar de obtenerlas de una fuente real:

        return new double[]{8.5, 7.2, 9.0, 6.8, 8.9};
    }
}

```

Ahora, podemos escribir una prueba unitaria para la clase `CalculadoraNotas` utilizando JUnit y nuestro stub `ServicioNotasStub`:

```

import org.junit.Test;
import static org.junit.Assert.assertEquals;

public class CalculadoraNotasTest {

    @Test
    public void testCalcularNotaMedia() {

        // Creamos una instancia de la CalculadoraNotas
        // y le pasamos el stub en lugar del ServicioNotas real.
        CalculadoraNotas calculadora = new CalculadoraNotas(new ServicioNotasStub());
        double notaMedia = calculadora.calcularNotaMedia();

        // Verificamos que la nota media sea la esperada:
        // (asumiendo dos decimales de error)
        assertEquals(8.28, notaMedia, 0.01);
    }
}

```

Este enfoque permite probar la funcionalidad de `CalculadoraNotas` de manera aislada sin depender del servicio de notas real y garantiza que la clase calcule correctamente la nota media en función de las notas simuladas del *stub*.

Para terminar con este tipo de pruebas hemos de señalar que las pruebas de integración, al igual que la mayoría de las pruebas, nunca han de hacerse en el entorno de producción de la aplicación. Siempre se debe preparar un entorno controlado lo más parecido al entorno real donde poder lanzar toda la batería de pruebas desarrolladas.

2.2.1. Automatizar pruebas de la GUI

Las pruebas de integración automatizadas pueden ser complicadas de aplicar si tomamos como punto de entrada la interfaz gráfica de nuestra aplicación. De forma programática podemos ir probando de forma unitaria o en conjunto partes o módulos internos de nuestra aplicación pero testear que la funcionalidad es robusta desde la propia interfaz de forma automática puede ser más complejo.

Necesitaremos una librería que nos permita invocar de forma programática los componentes gráficos incluidos en la GUI y analizar su respuesta, o bien, usar algún tipo de mecanismo de «grabación» para poder crear secuencias de interacciones (clicks de ratón, movimientos, etc.) y lanzar estas secuencias contra nuestra interfaz gráfica. Ambos mecanismos son costosos de realizar y pueden ser complejos.

Existen herramientas que pueden facilitarnos el trabajo en este sentido como por ejemplo: Selenium, Appium, WinAppDriver y Code UI Test. Es importante destacar que la elección de la herramienta adecuada para la automatización de pruebas de aplicaciones de escritorio dependerá de varios factores, incluido el sistema operativo, el lenguaje de programación preferido, la tecnología de la aplicación de escritorio y los requisitos específicos del proyecto.

2.3. Pruebas de regresión

Estas pruebas se centran en verificar que las nuevas actualizaciones, correcciones de errores o cambios en el código no afecten negativamente a las características existentes del software. A medida que se desarrolla y mejora el software, es fundamental mantener la integridad y el rendimiento de las características existentes. Las pruebas de regresión ayudan a asegurarse de que las modificaciones no introduzcan nuevos errores o provoquen la regresión de problemas previamente resueltos. Esto es crucial para garantizar que los usuarios no se vean afectados negativamente por las actualizaciones.

Cuando hacemos pruebas de regresión, estamos comprobando que el código que hemos modificado se comporta como queremos, y que el cambio no ha causado otros problemas en otros sitios del código, que funcionaban correctamente la última vez que los probamos. Estas pruebas de regresión se pueden hacer de forma manual, o automatizada, pero en muchos casos se suelen automatizar, para ejecutarse frecuentemente. Las pruebas de regresión suelen tener un papel importante en el proceso de integración. Este tipo de pruebas no necesitan ser escritas como tal, sino que se pueden crear a partir de pruebas que hemos ido elaborando durante el desarrollo. La idea es que cada vez que modificas el código fuente, ejecutes una selección de pruebas relevantes que ya tenías para ver si todo funciona como debería. Pruebas funcionales, pruebas unitarias, pruebas de integración (cualquier cosa que haya verificado durante el proceso de desarrollo que ciertos componentes del software funcionan como deberían) pueden actuar como pruebas de regresión.

2.4. Pruebas de estrés

Las pruebas de rendimiento y estrés del software son un componente crítico en el ciclo de desarrollo de aplicaciones, ya que permiten evaluar cómo una aplicación se comporta bajo diferentes condiciones de carga. Estas pruebas ayudan a identificar cuellos de botella, ineficiencias y problemas de rendimiento que podrían afectar la experiencia del usuario final.

Este tipo de pruebas son generalmente automáticas y casi siempre puentean o prescinden de cualquier interfaz gráfica de acceso a la aplicación. En lugar de eso, las herramientas usadas para la elaboración de estas pruebas suelen conectarse directamente a las capas interiores de nuestra aplicación a través de mensajes de red (HTTP, SOAP, etc.) y de esa forma probar de forma exigente como reaccionan estas capas sobre las máquinas que soportarán la infraestructura principal. Estas pruebas de rendimiento suelen medir tiempos de respuesta de bases de datos, discos, ocupación de memoria, etc. Todo ello suele medirse tras la creación de una alta demanda a través de la creación de múltiples peticiones paralelas en diferentes hilos o *threads*.

Las pruebas de estrés de software son comúnmente realizadas en etapas avanzadas, cuando los distintos módulos que conforman al sistema han sido integrados. Por ello suelen comenzar a realizarse tras varias iteraciones en el proceso general del producto, cuando ya hemos testeado la integración de los módulos principales. No son especialmente costosas pero llevan más tiempo de preparación y de análisis de resultados que otro tipo de pruebas.

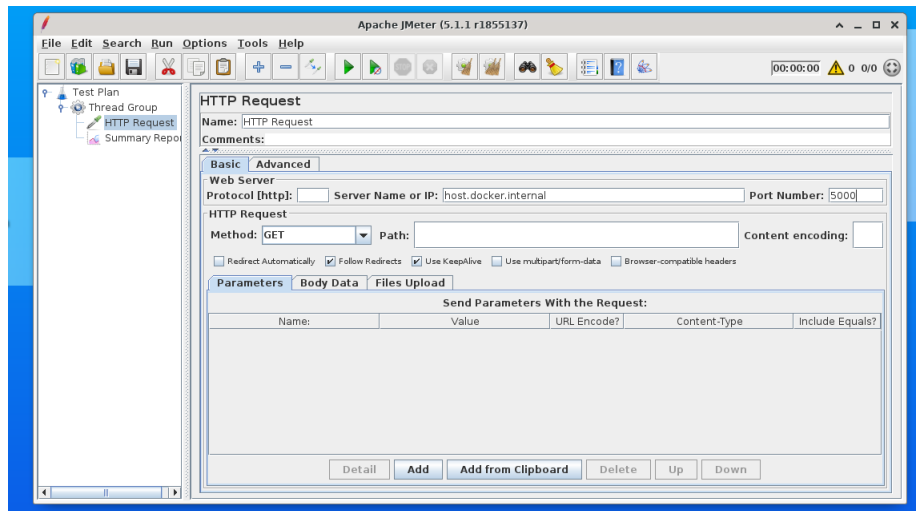


Figura 2: Ejemplo de uso de JMeter a través de su interfaz gráfica

Una de las herramientas más usadas en el ecosistema Java para este tipo de pruebas es JMeter. Una aplicación Java desarrollada por la fundación Apache y que podemos descargar de forma gratuita desde su web. El funcionamiento de JMeter es muy completo e intuitivo. Podremos ir configurando el lanzamiento de nuevos hilos y el tipo de petición que queremos que JMeter haga al servicio que estamos probando. Las pruebas de rendimiento con JMeter implican la creación de planes de prueba que describen el escenario de carga que se simulará. Estos planes de prueba pueden ser tan simples o complejos como lo requiera el proyecto. JMeter además nos ofrece un panel de análisis de los resultados obtenidos para su estudio posterior.

2.5. Pruebas de seguridad

Las pruebas de seguridad de un producto de software son un componente crítico en el proceso de desarrollo, ya que se centran en identificar y mitigar posibles vulnerabilidades y riesgos que podrían ser explotados por atacantes malintencionados. Estas pruebas se diseñan para evaluar la resistencia de un sistema o aplicación a amenazas de seguridad y para garantizar que los datos y la funcionalidad estén protegidos de manera adecuada. Las pruebas de seguridad, a menudo conocidas como pruebas de penetración o pruebas de hacking ético, son un proceso en el que se simulan ataques controlados para identificar vulnerabilidades y debilidades en un producto de software. El objetivo principal es descubrir posibles puntos de explotación que podrían ser utilizados por ciberdelincuentes para acceder, dañar o comprometer el sistema.

Antes de llevar a cabo las pruebas de seguridad, es importante recopilar información sobre el sistema, sus componentes, sus tecnologías y las amenazas potenciales. Esto ayuda a los evaluadores a comprender mejor el entorno y a diseñar pruebas específicas. Se determina qué áreas y funcionalidades se evaluarán y qué técnicas de prueba se utilizarán. Esto incluye pruebas de inyección SQL, pruebas de XSS (cross-site scripting) en el caso de servicios web y otras. Durante esta fase, los evaluadores llevan a cabo los ataques simulados, intentando explotar las vulnerabilidades detectadas. Es importante realizar las pruebas de manera ética y respetando la legalidad, sobre todo si hay otros servicios involucrados que puedan estar manejando información protegida de algún modo.

Después de las pruebas, se analizan los resultados en busca de vulnerabilidades y debilidades. Se identifican los riesgos y se documentan las áreas que requieren corrección o mitigación. Los resultados se documentan en un informe detallado que incluye descripciones de las vulnerabilidades encontradas, su gravedad y recomendaciones para su corrección. Esto proporciona a los desarrolladores la guía necesaria para abordar las vulnerabilidades. Los resultados se documentan en un informe detallado que incluye descripciones de las vulnerabilidades encontradas, su gravedad y recomendaciones para su

corrección. Esto proporciona a los desarrolladores la guía necesaria para abordar las vulnerabilidades.

3. Documentación y plan de pruebas

Un **plan de pruebas** es un documento esencial que describe en detalle cómo se llevarán a cabo las pruebas de una aplicación o sistema de software. El objetivo principal de un plan de pruebas es establecer un enfoque estructurado y sistemático para evaluar el software y garantizar que cumple con los requisitos y estándares de calidad establecidos. A continuación, te proporcionaré una descripción general de lo que implica un plan de pruebas y cómo documentarlo adecuadamente:

- Objetivos de las Pruebas: En esta sección, se describen los objetivos específicos que se persiguen con las pruebas. Esto puede incluir la validación de requisitos funcionales, el aseguramiento de la calidad, la identificación de defectos, entre otros.
- Criterios de Aceptación: Especifica los criterios que se utilizarán para determinar si una prueba se considera exitosa o no. Esto a menudo incluye umbrales de rendimiento, requisitos de cumplimiento y estándares de calidad.
- Recursos Necesarios: Enumera los recursos necesarios para llevar a cabo las pruebas, como hardware, software, personal, datos de prueba y entornos de prueba.
- Planificación y Programación: Describe la programación de las pruebas, incluyendo cuándo se llevarán a cabo, la duración prevista y la secuencia de las pruebas.
- Estrategia de Pruebas: Detalla la estrategia general que se utilizará para realizar las pruebas. Esto puede incluir pruebas unitarias, pruebas de integración, pruebas de sistema, pruebas de aceptación del usuario, etc.
- Procedimientos de Pruebas: Define los pasos específicos que los evaluadores seguirán al realizar las pruebas. Esto puede incluir casos de prueba, secuencias de interacción, datos de prueba y escenarios de uso.
- Entregables: Enumera los informes y documentos que se generarán como resultado de las pruebas, como informes de resultados de pruebas, registros de errores y documentación de procedimientos.
- Roles y Responsabilidades: Define quiénes son responsables de llevar a cabo las pruebas, revisar los resultados y tomar decisiones basadas en los hallazgos.

Respecto al último punto referido a los **roles encargados del plan de pruebas** podemos añadir que, aunque estos roles pueden variar de organización a organización hay siempre algunos en común. En equipos pequeños, es común que los desarrolladores participen activamente en la creación de pruebas, mientras que en equipos más grandes, se pueden tener roles específicos de especialistas en pruebas y líderes de pruebas dedicados. La elección del modelo de pruebas y la asignación de responsabilidades dependen de la estructura y la cultura de la organización de desarrollo de software. Normalmente, en equipos grandes solemos tener subequipos orientados solo a las pruebas. Es posible que estén liderados un organizados entorno al departamento de «Calidad de Producto» o algún otro nombre similar.

Lista de ejemplos

1	Ejemplo de JUnit (di-029)	5
---	---------------------------	---

Actividades

1. Crea una clase `punto` que reciba dos números enteros y represente un punto cartesiano en un espacio de dos dimensiones. Implementa un método llamado `distancia` que calcule la distancia entre dos puntos. A partir de la clase `punto`, crea una clase `rectángulo` que reciba dos puntos, el que representa su esquina superior izquierda y el que representa su esquina inferior derecha. Implementa en `Rectángulo` los métodos `area()` y `perímetro()`. Una vez tengas todo implementado realiza los test necesarios para asegurarte de que las dos clases están bien programadas.
2. Tenemos una clase `Carrera`, referida a una carrera ciclista. Una `Carrera` puede ser de tres tipos de deporte (natación, ciclismo, atletismo) y además tendrá una distancia recorrida en metros y un tiempo (en segundos) que ha utilizado el ganador para terminarla. La carrera tendrá un método para calcular el ritmo medio del ganador (tiempo medio empleado en 1000 metros de actividad) y realiza los test necesarios para asegurarte de que la clase está bien implementada.
3. Añade al ejercicio anterior una clase `Campeonato` que consista en un conjunto de carreras. Un `Campeonato` debe permitir:
 - Añadir carreras
 - Retornar la carrera más larga, indicando el deporte
 - Retornar la carrera de mayor duración indicando el deporte
 - Retornar la carrera más rápida (menor ritmo) indicando el deporte

Añade esta clase a la clase `Carrera` que has hecho anteriormente y realiza los tests necesarios para asegurarte de que la clase está bien implementada.

4. Utiliza una clase llamada `Alumno` que modele un alumno tipo junto con su nombre y su edad como número entero. Supón que en tu código tienes una clase de acceso a los datos llamada `AulaReal` que, entre otras cosas, recupera alumnos de la base de datos. Esta clase implementa una interfaz llamada `Aula` similar a:

```
public interface Aula {  
    List<Alumno> obtenerTodosLosAlumnos();  
}
```

Construye ahora un caso de prueba donde verifiques el buen funcionamiento de varios objetos alumnos recuperados a través de un stub que implemente esta interfaz anterior y que simule ser la clase real que usa la base de datos.